

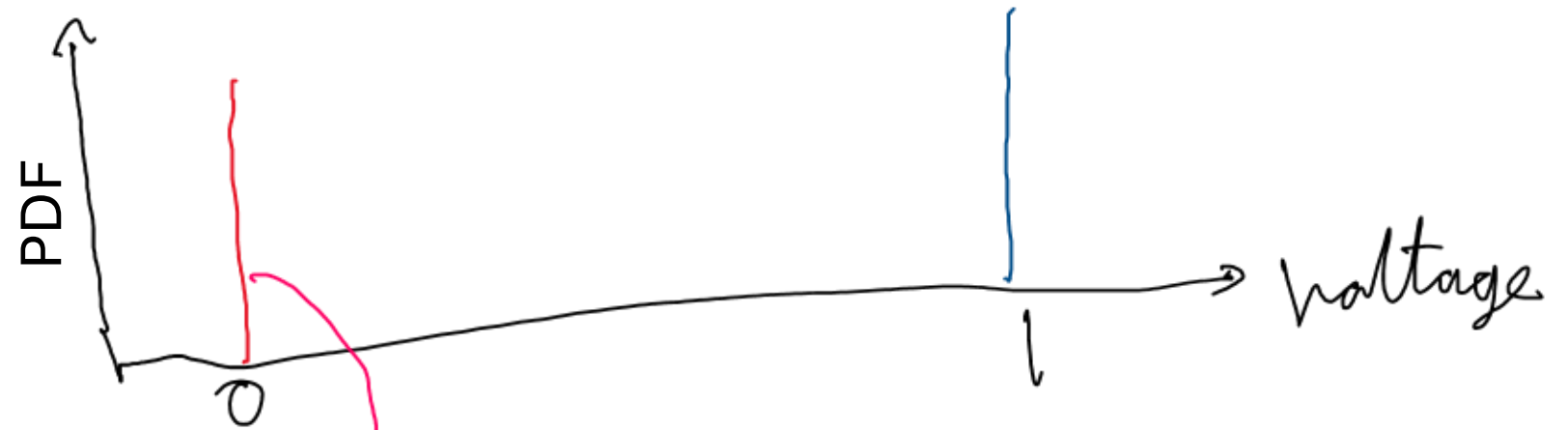
Lecture 9: Physical layer - Error detection and reliable transmission

Lecturer: Venkat Arun

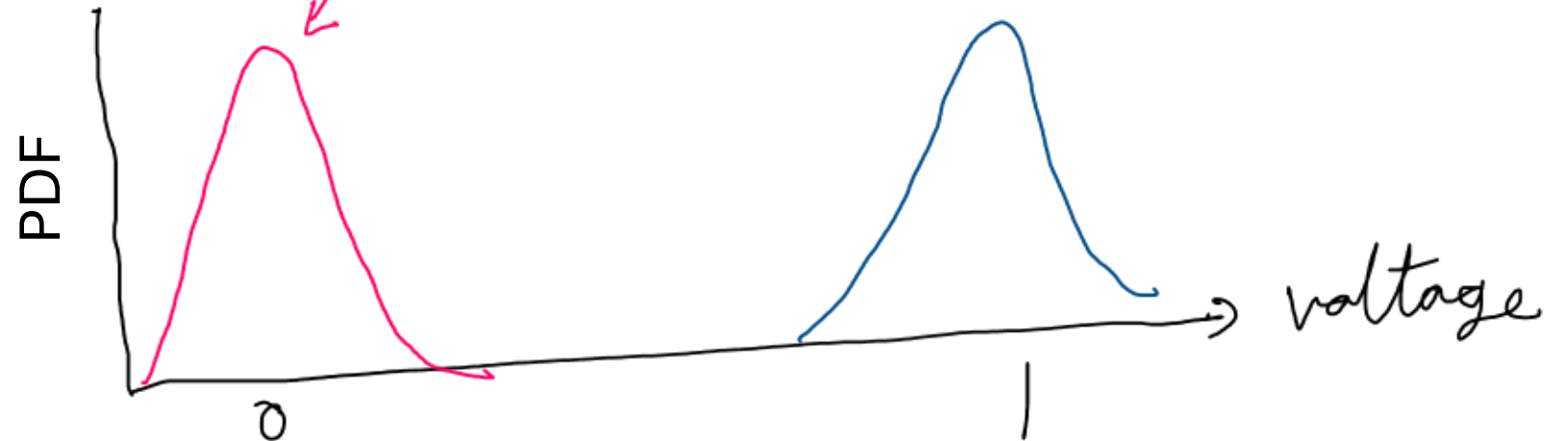
Some slides borrowed from Daehyoek Kim

Simple model for why errors happen

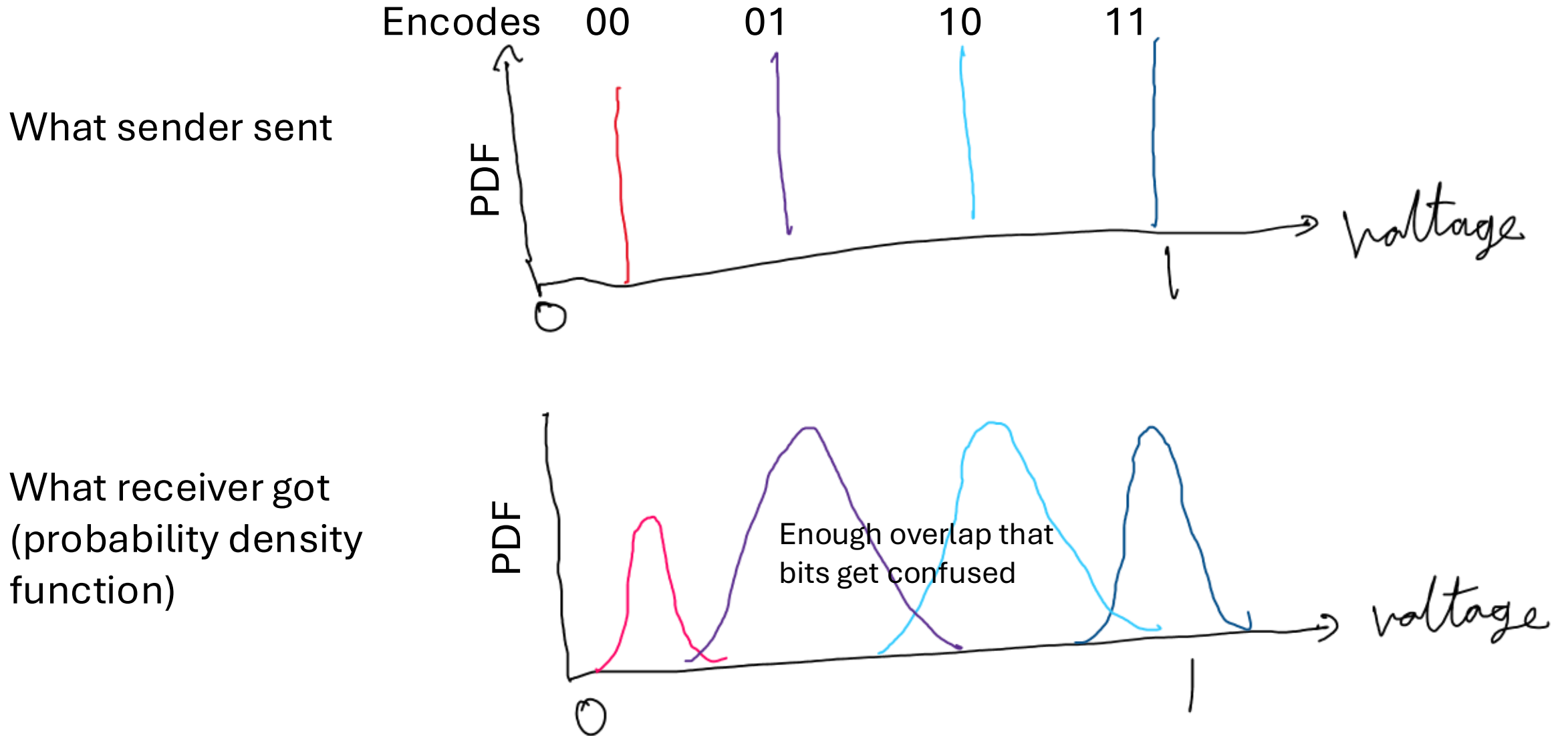
What sender sent



What receiver got
(probability density
function)



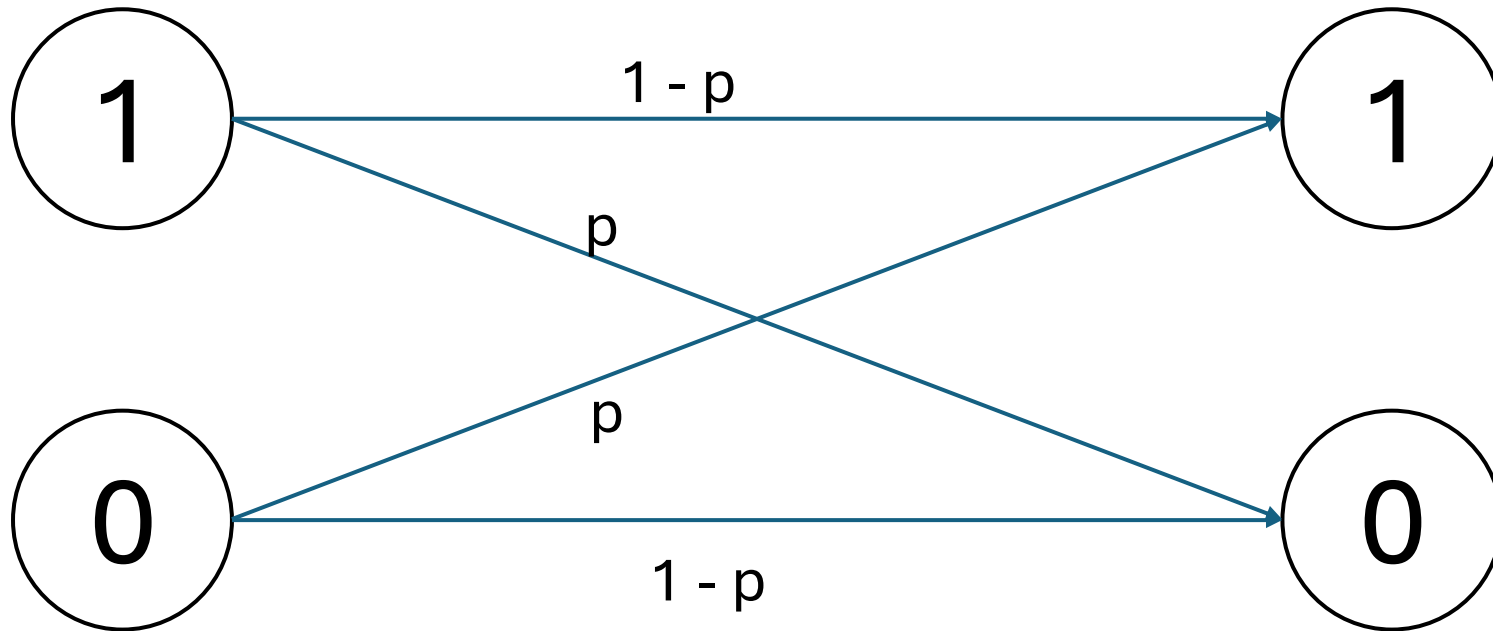
Simple model for why errors happen



What errors look like in the digital world

Bit that is sent

Bit that is received

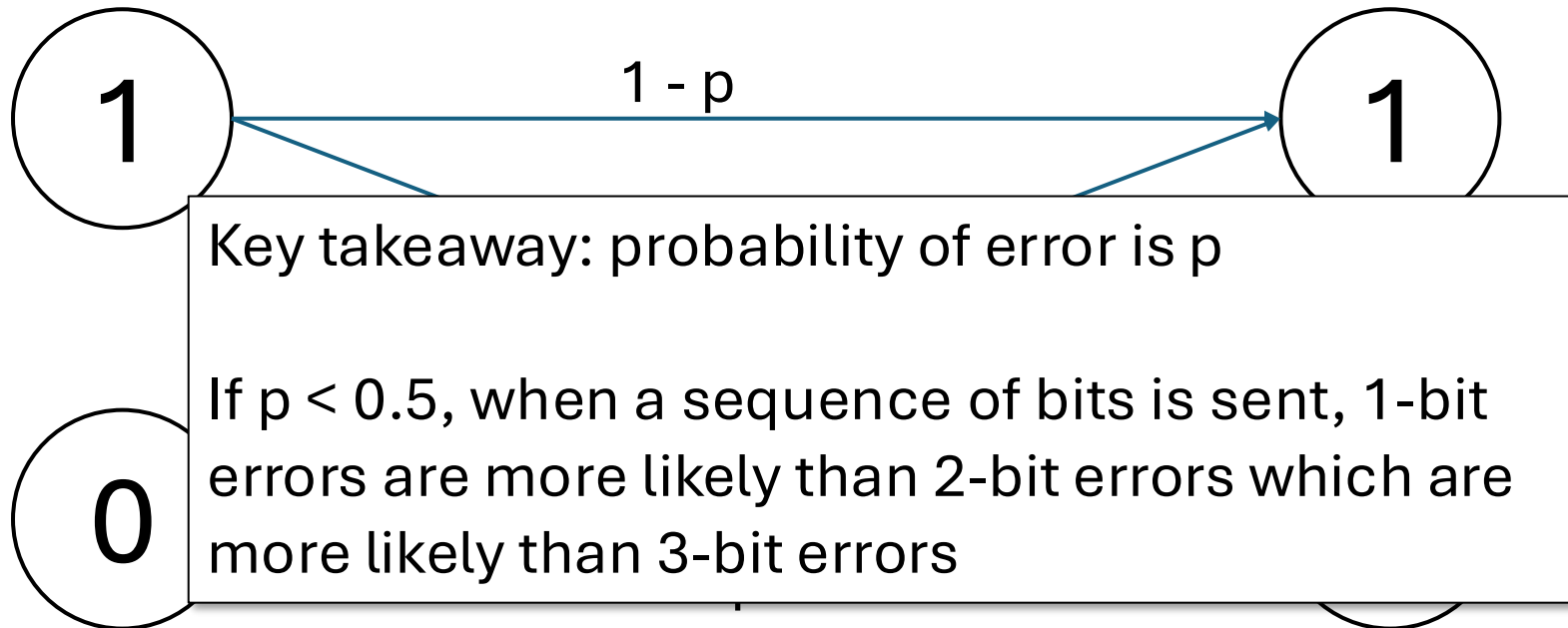


The values on the edges denote the probability that the transmitted bit is received as a 0 or a 1

What errors look like in the digital world

Bit that is sent

Bit that is received



The values on the edges denote the probability that the transmitted bit is received as a 0 or a 1

Error detection (typo version)

If you see one of the words below in an English text, you know it is a mistake:

cxt

bime

tast

Error detection (typo version)

If you see one of the words below in an English text, you know it is a mistake:

cxt cat, cot, cut

bime time, dime, lime, bite

tast fast, last, past, test, cast

Some English word errors cannot be detected this way:

sore core, bore, sire

stationery stationary

histerical historical, hysterical

However, there are multiple possible corrections. You cannot be certain which one it is

Error correction (typo version)

For the following words, there is a unique word (afaict) that is attained by modifying exactly one letter. In some cases it is reasonable to assume that this word is the most likely substitute.

virst first

seperate separate

elephent elephant

For these words, there is a unique English word that is exactly one letter off.

Note, most of these have to be longer. Why?

Benefit of length (typo version)

Longer words make both detection and correction easier. And we can make sentences arbitrarily long.

“The car remained stationery” -> stationary

“I bought some stationery” -> word is correct, no correction needed

Error model (typo version)

In examples so far, we have considered what happens when one letter is replaced by another. Other types of errors exist:

caht → cat, chat

wold → would, world, old

teh → meh, the

Hamming distance

Recall our error model. In many digital communication systems, it is reasonable to assume that fewer bit flips are more likely

This is not always true. E.g. sometimes an entire sequence of bits are set to 1. These are called burst-errors and occur due to spikes of electrical interference. We will not study these in this course.

In a bit-flip model, we measure distance using the hamming distance between two n -bit strings x and y :

$H(x, y)$ = # of bits that are different between x and y

E.g. $H(0100, 0100) = 0$, $H(0100, 0110) = 1$, $H(0100, 0111) = 2$

Mental model for error detection and correction

Error detection

Define a set of words (n-bit strings) called the dictionary D of n-bit strings

When we receive $x \in \{0,1\}^n$

Measure of quality: what is the largest integer k such that:

$$\forall x, y \in D, H(x, y) > k$$

Error correction

Use the same kind of dictionary D with the same k

Say we receive $x \in \{0,1\}^n$

$$\text{If } \exists y \in D, H(x, y) \leq \text{floor}\left(\frac{k}{2}\right)$$

$$\text{then } \forall z \in D, H(x, z) > H(x, y)$$

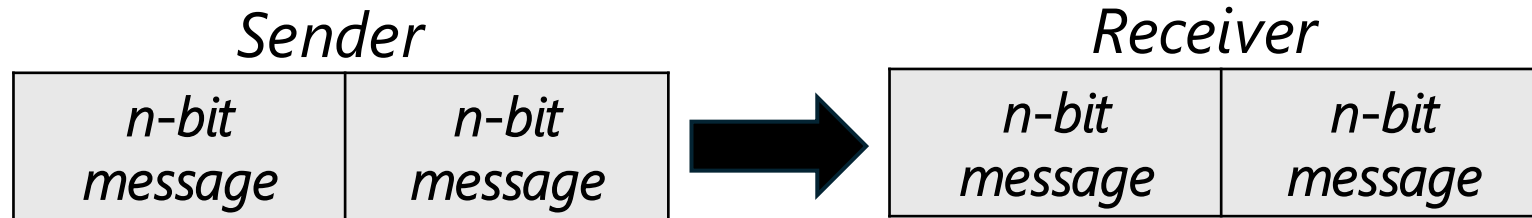
Therefore y is the most likely correction

Basic idea of error detection

Adding redundant information to a frame that can be used to determine if errors have been introduced

Naïve approach: Transmitting two complete copies of data

- Identical → No error
- Differ → Error



Is this good enough? If not, why?

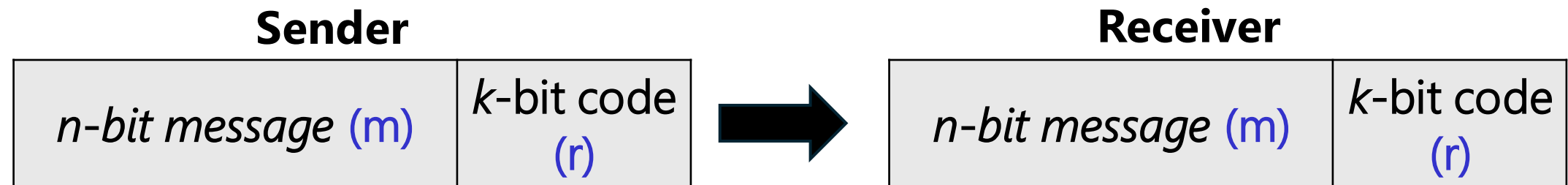
- n redundant bits for an n -bit message
- Error can go undetected – possibility to corrupt the same bit positions in the first and second copies of the message

More efficient error detection methods

k redundant bits for n bits message, where $k \ll n$

How to construct k -bit redundant information?

- Add **no new information** to the message
- **Derived from the original message** using some algorithm
- **Both the sender and receiver know the algorithm**



Receiver computes r using m
If they match, no error

Error detection – parity bits

- If I send some bits and they get corrupted on the way, say due to physical noise, what happens?
- We can add extra “error detection” bits to ensure that we at least find out that this has happened
- Simplest error detection mechanism is the parity check bit.

Rule: the final bit is the **parity bit** ensures that there are always an even number of 1s

Examples:

- 0000 → 0000 **0**
- 1011 → 1011 **1**
- 0100 → 0100 **1**

If one bit is flipped, we can detect it

Error detection - checksum

- How do we detect more bit errors? One method is to view the transmitted bits as a sequence of k-bit integers
- To the message of size $n = m * k$, add another k bits such that the sum of all the k-bit integers in the message is 0

Example:

$k = 8$

Message: [10, 237, 213]

$(10 + 237 + 213) \% 256 = 204$

Now $204 + 52 = 256$

Thus, we can append 52 to get the encoded message: [10, 237, 213, 52]

Error detection - checksum

How good is checksum?

- It catches all single-bit errors and most multi-bit errors.
- It cannot catch *all* 2-bit errors. E.g. if we flipped the least-significant bits of two 8-bit integer, the sum remains the same
- Better error detection mechanisms have been designed to detect specific types of errors that are common in practice
- The book describes CRC, which catch catch the following (not a part of this course)
 - All single- and double-bit errors
 - All odd-bit errors (and hence all triple-bit errors)
 - Any burst error less than k-bits long
 - Most other multi-bit errors

Example of an error correcting code: triple repetition

Data I want to transmit	How I encode the data
0	000
1	111

What I receive	What I interpret it as
000	0
111	1

Example of an error correcting code: triple repetition

Data I want to transmit	How I encode the data
0	000
1	111

What I receive	What I interpret it as
000	0
111	1
001	??

Example of an error correcting code: triple repetition

Data I want to transmit	How I encode the data
0	000
1	111

What I receive	What I interpret it as
000	0
111	1
001	0
011	??

Example of an error correcting code: triple repetition

Data I want to transmit	How I encode the data
0	000
1	111

What I receive	What I interpret it as
000	0
111	1
001	0
011	1
101	1
100	0

A longer error correcting code

Data I want to transmit	How I encode the data (codeword)
0000	0000000
0001	1101001
0010	0101010
0011	1000011
0100	1001100
0101	0011001
0110	1100110
0111	0001111
1000	1110000
1001	0100101
1010	1011010
1011	0110011
1100	0111100
1101	1010101
1110	0010110
1111	1111111

Points to note

- Every bit-string on the right differs from every other bit-string by at least 3 bits. This is called as having a “hamming distance” of at least 3
- This means that if one bit of any codeword is flipped, its distance to every other codeword is ≥ 2 . Its distance to the original codeword is of course 1. Thus, it can be decoded
- If two bits are flipped, it cannot be **corrected**, but the error can be **detected** since no two codewords differ by 2 bits
- How do we come up with this table? This is called a hamming code and is out of scope of this course. It is simple enough to understand though

A longer error correcting code

Data I want to transmit	How I encode the data (codeword)
0000	0000000
0001	1101001
0010	0101010
0011	1000011
0100	1001100
0101	0011001
0110	1100110
0111	0001111
1000	1110000
1001	0100101
1010	1011010
1011	0110011
1100	0111100
1101	1010101
1110	0010110
1111	1111111

Here, we are encoding 4 bits as 7 bits

In general, for all m , you can encode $2^m - m - 1$ bits using a codeword of $2^m - 1$ bits. In each case, you can correct 1 bit errors and detect 2 bit errors

For example:

1 bit $\rightarrow$ 3 bits (efficiency: 0.333, $m = 2$)

4 bits $\rightarrow$ 7 bits (efficiency: 0.571, $m = 3$)

11 bits $\rightarrow$ 15 bits (efficiency: 0.733, $m = 4$)

26 bits $\rightarrow$ 31 bits (efficiency: 0.839, $m = 5$)

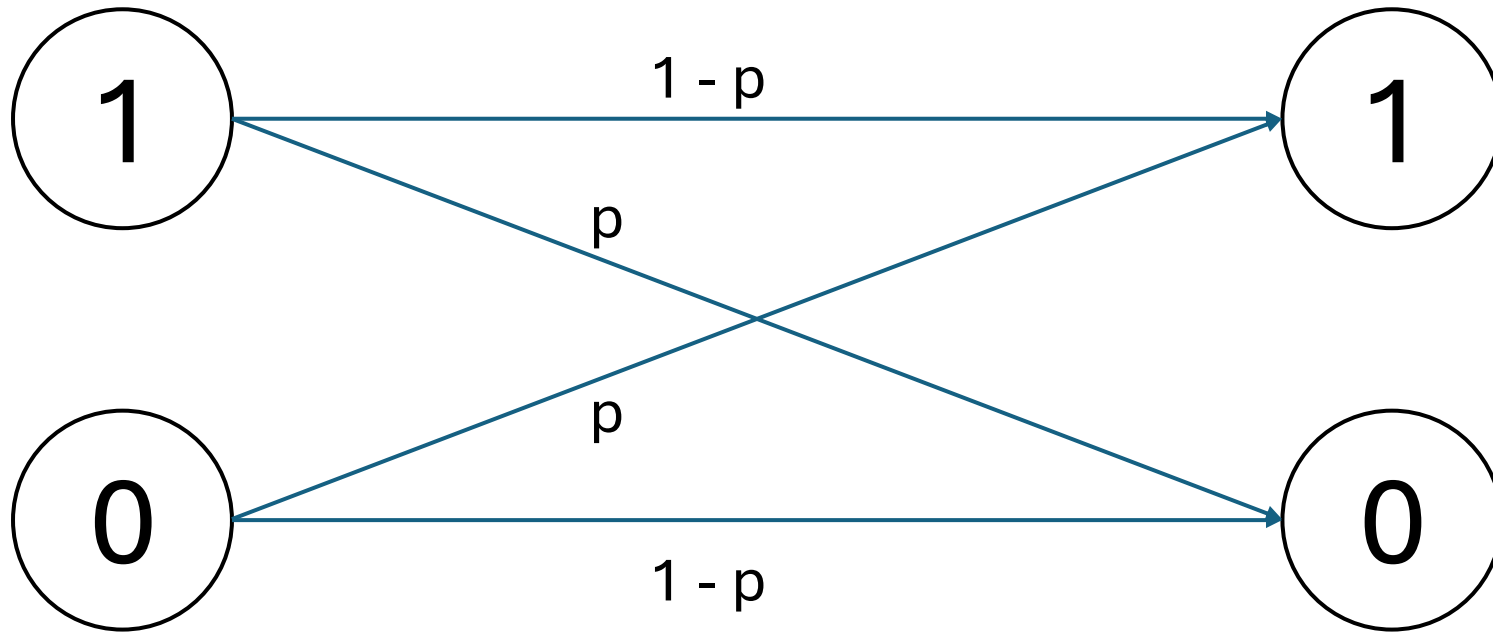
Key takeaway: To correct/detect a given number of bit errors, the longer codewords you use, the greater the efficiency.

Efficiency approaches 1 as codeword length goes to infinity!

Multi-bit correction: How much error correction do we need?

Bit that is sent

Bit that is received



If probability of error is p , then we will have $\approx Kp$ errors in every K bits

Magic of statistics: as $K \rightarrow \infty$, the number of errors will be almost *exactly* Kp

In case you are interested, this is called the law of large numbers

Multi-bit correction: How much error correction do we need?

If probability of error is p , then we will have $\approx Kp$ errors in every K bits

Magic of statistics: as $K \rightarrow \infty$, the number of errors will be almost *exactly* Kp

In case you are interested, this is called the law of large numbers

Task: To transmit N bits, we need to find a $K > N$ bit code book (set of codewords) that encodes K bits such that we can correct Kp . Efficiency is N / K

Shannon's theorem (informally stated): For every channel with probability of error p , we can construct bigger and bigger code books (i.e. larger and larger N and K) such that the efficiency approaches $1 - H(p)$. That is, $\frac{N}{K} \rightarrow 1 - H(p)$ as $N \rightarrow \infty$

Here, $H(p) = -p \log p - (1 - p) \log(1 - p)$ is called “information entropy”

Multi-bit correction: How much error correction do we need?

Task: To transmit N bits, we need to find a $K > N$ bit code book (set of codewords) that encodes K bits such that we can correct Kp . Efficiency is N / K

Shannon's theorem (informally stated): For every channel with probability of error p , we can construct bigger and bigger code books (i.e. larger and larger N and K) such that the efficiency approaches $1 - H(p)$. That is, $\frac{N}{K} \rightarrow 1 - H(p)$ as $N \rightarrow \infty$

Here, $H(p) = -p \log p - (1 - p) \log(1 - p)$ is called “information entropy”

Strategy: Pick a very large K and pick $N = (1 - H(p))K$

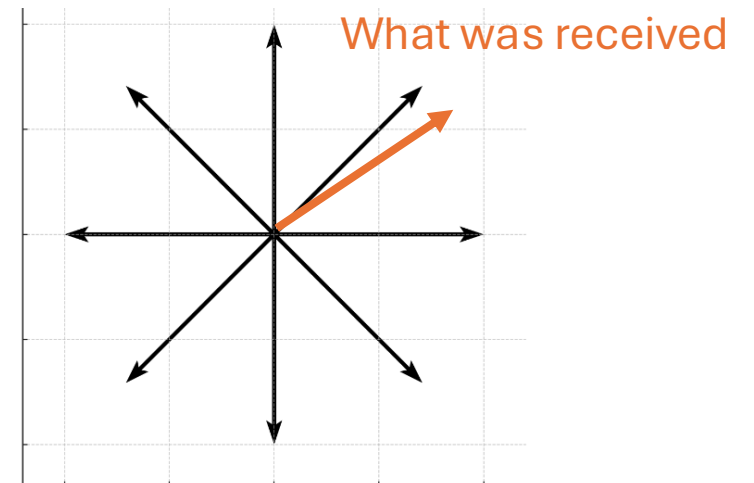
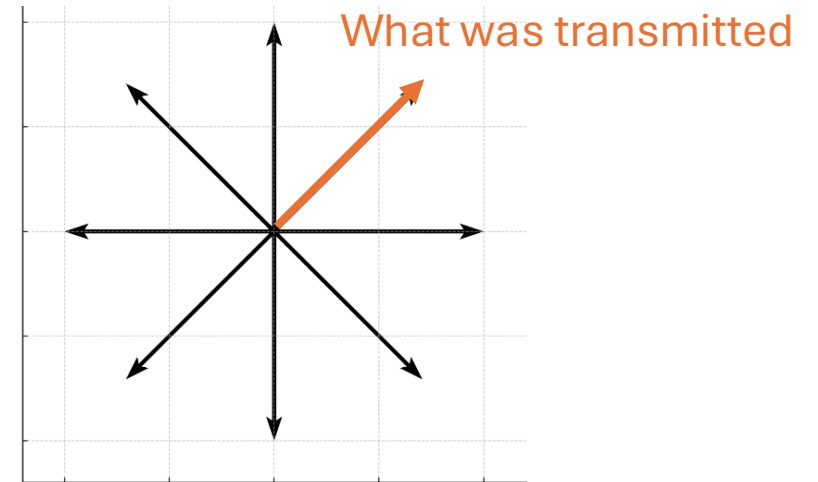
Key takeaways

The following is all I expect you to understand about multi-bit error coding:

- We can encode messages by their corresponding codewords
- A bit-flip channel flips the bit with probability p
- For p , there is a maximum achievable efficiency for messages sent through that channel.
- That is, we can encode N -bit messages using K -bit codewords such that N / K approaches this maximum as $N \rightarrow \infty$

Fun aside: Analogy between neural networks and Shannon's code

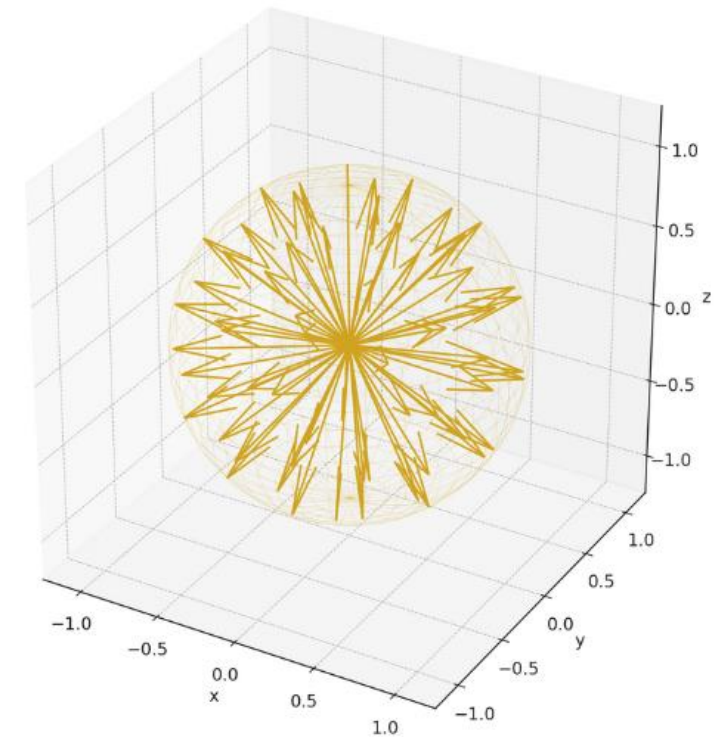
- Say we transmit vectors and what is received is the transmitted vector rotated by a little bit (this is not how physical channels work, but in high dimensions, it is close enough)
- The transmitter encodes information in one of the 8 available vectors ($\log_2 8 = 3$ bits)
- The receiver corrects to the nearest vector. If the noise is less than 22.5° , the result will be correct



Fun aside: Analogy between neural networks and Shannon's code

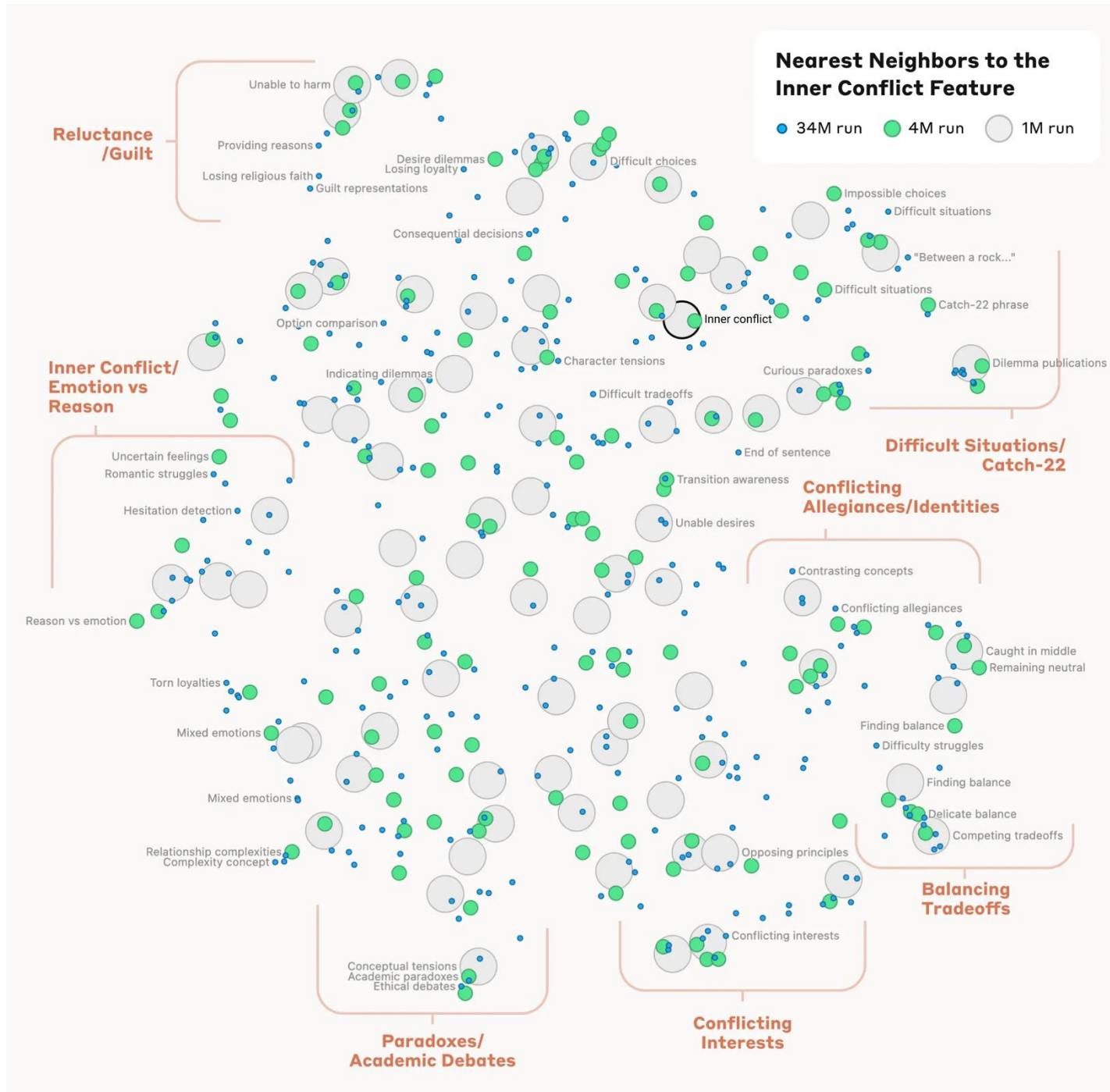
- In higher dimensions, you can fit more vectors while still making sure the angle between any two vectors is larger than the noise.
- In general, in a d -dimensional space, you can pick $O(2^d)$ unit vectors (and hence encode $O(d)$ bits) such that the distance between any two of them is $O\left(\frac{1}{\sqrt{d}}\right)$
- How do we pick these vectors? Easy, just pick randomly!

Approximately Max-Min Angle Vectors on S^2 ($N=64$, min angle $\approx 22.27^\circ$)



Fun aside: neural networks

- Many neural networks have been shown to represent concepts as directions in a high dimensional vector space.
- Here is an excellent study by Anthropic that interprets LLMs' internal representation using this idea:
<https://www.anthropic.com/research/mapping-mind-language-model>
- If the vector has $O(d)$ dimensions, the network can represent $O(2^d)$ concepts!
- Anthropic found directions corresponding to many high-level concepts (see right)



Fun aside: Shannon's code

- A similar result is true if, instead of using real numbers, the vectors are in the field Z_2 i.e., all elements are binary.
 - A vector is a bit-string. The sum of two vectors is a bit-wise XOR of the two strings.
 - The distance (dot product in the previous slide) between two vectors is the Hamming distance between them (i.e. the number of positions where the bit strings are different)
- Thus, you can have a large number of vectors (bit-strings) that have a large hamming distance between them (say X). This is exactly what we needed for error correction. As long as $< X/2$ bits are flipped, we can correct the error
- The larger K needs to be, the fewer vectors (bit-strings) you can pack before you get a pair that differ by $< X$ bits.

Recap: How do we correct more bit errors?

- *Shannon's code*: For every N -bit message, pick a *random* K -bit string as the codeword. Let this mapping from message to codeword be $C(m)$. The sender and receiver need to agree on this function
- Decoding strategy: given a received value of x , find a message m such that the hamming distance between x and $C(m)$ is minimized. m will be the most likely transmitted message
- What is the computational complexity of this decoding process?

Fun aside: How do we correct more bit errors?

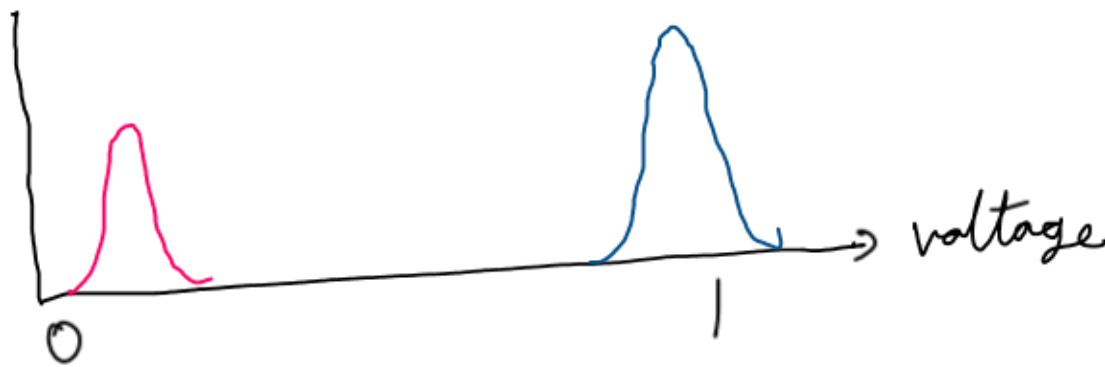
- What is the computational complexity of this decoding process?
- Possible strategy to decode received message x :
 - I received x
 - Is $C^{-1}(x)$ a message? If so, return $C^{-1}(x)$
 - For all integers i
 - For all x' such that hamming distance between x and x' is i Complexity: there are 2^i such x' !
 - Is $C^{-1}(x')$ a message? If so, return $C^{-1}(x')$

Key takeaways and some history

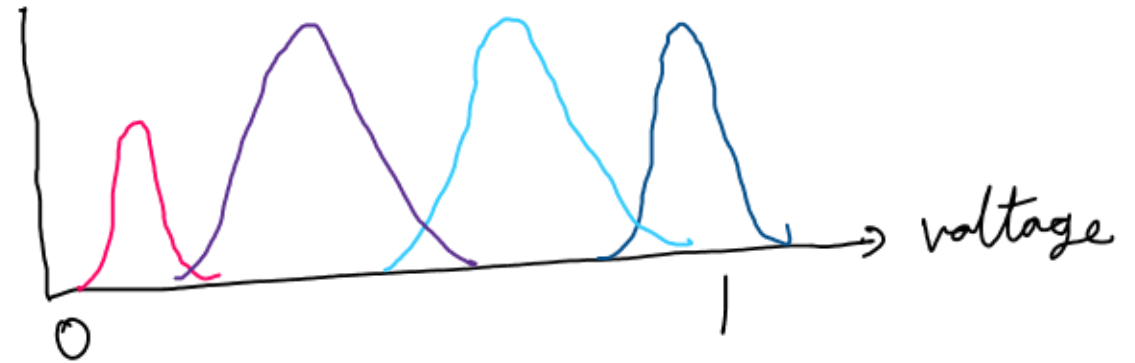
- Decoding Shannon codes has exponential complexity
- Shannon published his ground-breaking work, including the Shannon code, in 1948
- However, for a long time we did not have a coding scheme that approached the theoretical limit *and* could be decoded efficiently (note the two different types of efficiency here).
- In the 1990s, we started studying the first solutions to this problem (called Turbo codes and LDPC codes)
- This revolutionized cellular and WiFi networks because, for the first time, they were able to approach the theoretical optimal message transmission rates! This was quickly adopted everywhere
- Side note: Turbo codes were patent protected until a few years ago. This made LDPC codes more popular in some places

Key takeaways and some history

Thanks to these new coding schemes, we could start packing bits more tightly. This way, errors are more likely to happen, but we now have efficient methods to correct them. This is what allowed wireless networks to achieve greater efficiency



More conservative encoding that prevented errors



Tighter, more aggressive encoding